



Projet GuestHub – Gestion des visiteurs

Réalisé par : Fabrice Orgrick Kouonang

Projet scolaire – Septembre 2025

Supervisé par : Kerlyne Fostiné,
Enseignante – Gérald Godin



U L T R A S P E C
A E R O P A R T M E T A L F I N I S H I N G

Table of Contents

1. INTRODUCTION.....	3
Objectifs.....	4
Portée.....	4
Identification du système.....	4
Fonction principale et limites.....	4
Contexte et intégration.....	5
Objectifs du projet.....	5
Bénéfices et retombées.....	5
Définitions.....	6
Documents de références.....	7
2. DESCRIPTION GÉNÉRALE DU LOGICIEL.....	8
Vue d'ensemble des fonctions du produit.....	8
1. Enregistrement des visiteurs.....	8
2. Gestion des badges.....	8
3. Check-in / Check-out des visites.....	8
4. Gestion des utilisateurs et rôles.....	8
5. Consultation et exportation des rapports.....	8
Interfaces externes.....	8
Spécifications fonctionnelles.....	9
Technologies utilisées.....	10
Exigences logiques de bases de données.....	10
Architecture du logiciel.....	13
3. DESCRIPTION DÉTAILLÉE.....	14
A. Exigences fonctionnelles.....	14
B. Exigences non fonctionnelles.....	16
Spécifications fonctionnelles.....	17
Exigences logiques de bases de données.....	17
Contraintes de conception.....	18
Exigences non-fonctionnelles.....	19
4. INFORMATIONS COMPLÉMENTAIRES.....	20
4.2 Quelques interfaces utilisateurs.....	20
Mobile Sur SwiftUi-----Invité Ipad.....	23
GABARIT DE PRÉSENTATION DES CAS D'UTILISATION.....	24

1. INTRODUCTION

Le présent document décrit l'ensemble des exigences fonctionnelles et non fonctionnelles du système **GuestHub**. Ce projet vise à concevoir une application de **gestion des visites** permettant de simplifier, sécuriser et automatiser le processus d'accueil des visiteurs dans une organisation.

GuestHub offre une plateforme numérique pour enregistrer les visiteurs, gérer leurs informations, suivre les entrées et sorties, et générer des rapports de fréquentation par employés et département.

Le but principal du système est d'améliorer l'efficacité du processus d'accueil, de réduire les erreurs manuelles et de renforcer la traçabilité des visites.

Ce document présente :

- Une vue d'ensemble du système à développer ;
- Les objectifs et la portée du projet ;
- Les exigences fonctionnelles détaillant les principales fonctionnalités ;
- Les exigences non fonctionnelles liées à la performance, la sécurité et l'ergonomie.

L'objectif est de fournir une base claire et complète pour le développement, la validation et la maintenance du système **GuestHub**.



Objectifs

L'objectif de ce document est de définir de manière claire et détaillée toutes les exigences du système **GuestHub**.

Il servira de référence tout au long du cycle de développement afin d'assurer que le produit final réponde aux besoins fonctionnels et techniques attendus.

Plus précisément, ce document vise à :

- Décrire les fonctionnalités principales de l'application guesthub ;
- Préciser les besoins des utilisateurs et les contraintes du système ;
- Établir une base commune de compréhension entre les différentes parties prenantes (développeurs, gestionnaires, utilisateurs finaux) ;
- Servir de guide pour la validation et les tests du produit.

Ce document s'adresse à l'ensemble des acteurs impliqués dans le développement, la mise en œuvre et l'utilisation du système **GuestHub**, notamment :

- **L'équipe de développement** : pour comprendre les fonctionnalités à implémenter et les contraintes techniques.
- **Les responsables ou gestionnaires de l'organisation** : pour valider que le système répond aux besoins de gestion des visiteurs.
- **Les utilisateurs finaux (réceptionnistes, personnel d'accueil)** : pour connaître les fonctionnalités disponibles et le flux d'utilisation prévu.
- **L'équipe de test et d'assurance qualité** : pour élaborer les scénarios de test et valider les exigences.
- **Toute autre partie prenante** impliquée dans la maintenance, le déploiement ou la documentation du système.

Portée

Identification du système

Le système logiciel, nommé **GuestHub**, est une application de **gestion des visites** destinée aux organisations souhaitant moderniser leur processus d'accueil. GuestHub permet de centraliser l'ensemble des informations liées aux visiteurs, de suivre les entrées et sorties en temps réel, et de faciliter la communication entre les visiteurs et les employés qu'ils viennent rencontrer.

Fonction principale et limites

GuestHub a pour rôle principal de :

- Enregistrer les visiteurs de manière rapide et sécurisée ;
- Notifier automatiquement les employés concernés à l'arrivée d'un visiteur ;
- Gérer le flux des visites en affichant leur état (en attente, en cours, terminée) ;
- Produire des rapports et statistiques sur la fréquentation.

Le système **ne vise pas** à :

- Gérer la totalité des opérations internes de l'organisation (par exemple : gestion du personnel, planification des salles, sécurité physique des bâtiments) ;
- Se substituer aux systèmes de contrôle d'accès ou de sécurité existants ;
- Effectuer une reconnaissance d'identité biométrique (hors portée de cette version).

Contexte et intégration

GuestHub s'intègre dans le contexte d'une organisation ou d'une entreprise recevant régulièrement des visiteurs — tels que des clients, fournisseurs, partenaires ou candidats. Actuellement, la plupart des accueils se font encore à l'aide de formulaires papier ou de feuilles de registre, entraînant une perte de temps et une traçabilité limitée. L'application vise donc à **numériser et automatiser** ce processus.

Objectifs du projet

Les objectifs principaux du projet **GuestHub** sont :

- **Améliorer l'efficacité** du processus d'accueil des visiteurs ;
- **Réduire les délais d'attente** à la réception ;
- **Assurer la traçabilité complète** des visites pour des raisons de sécurité et de conformité ;
- **Faciliter la communication** entre les visiteurs et le personnel interne ;
- **Offrir une interface moderne et intuitive** adaptée à différents appareils (tablettes, ordinateurs, mobiles).



Bénéfices et retombées

La mise en place de **GuestHub** apportera plusieurs avantages tangibles :

- Diminution des erreurs de saisie et des pertes d'informations ;
- Réduction du temps consacré à la gestion manuelle des registres de visiteurs ;
- Amélioration de l'image professionnelle de l'organisation grâce à un accueil numérique et fluide ;
- Meilleure sécurité et meilleure visibilité sur la présence des visiteurs dans les locaux.

Définitions

Cette section présente les principaux termes utilisés dans le cadre du projet **GuestHub** afin d'assurer une compréhension commune du vocabulaire.

Terme / Acronyme	Définition
GuestHub	Nom du système logiciel de gestion des visites développé pour automatiser l'accueil et le suivi des visiteurs.
Visiteur	Toute personne externe à l'organisation (client, fournisseur, partenaire, candidat, etc.) qui accède temporairement aux locaux.
Employé hôte	Employé de l'organisation recevant le visiteur. Il est notifié lors de l'arrivée de son invité.
Accueil / Réceptionniste	Utilisateur responsable de l'enregistrement et de la validation des informations des visiteurs.
Badge / Code QR	Moyen d'identification temporaire attribué à un visiteur pour la durée de sa visite.
Session de visite	Période comprise entre l'arrivée et le départ d'un visiteur, enregistrée dans le système.
Interface utilisateur (UI)	Partie visible de l'application permettant à l'utilisateur d'interagir avec le système.
Base de données	Système de stockage des informations relatives aux visiteurs, aux employés et aux historiques de visites.
Notification	Message automatique envoyé à l'employé concerné pour l'aviser de l'arrivée d'un visiteur.
CRUD	Acronyme de <i>Create, Read, Update, Delete</i> , désignant les quatre opérations de base sur les données.
QR (Quick Response)	Type de code à barres bidimensionnel utilisé pour identifier rapidement un visiteur ou une session.
API (Application Programming Interface)	Interface de programmation permettant la communication entre différentes applications ou services.
S3 (Amazon Simple Storage Service)	Service de stockage d'objets fourni par Amazon Web Services (AWS) utilisé pour héberger des fichiers et images.
AWS (Amazon Web Services)	Plateforme de services cloud utilisée pour l'hébergement et l'infrastructure du système.
PC (Personal Computer)	Ordinateur personnel utilisé pour accéder à la version web de GuestHub.
App Store	Plateforme de distribution d'applications pour les appareils Apple.
Play Store	Plateforme de distribution d'applications pour les appareils Android.
SwiftUI	Framework de développement d'interfaces utilisateur pour les applications iOS.
Node.js	Environnement d'exécution JavaScript côté serveur utilisé pour le backend.
PostgreSQL	Système de gestion de base de données relationnelle utilisé pour stocker les informations du système.
Keycloak	Solution d'authentification et de gestion des rôles des utilisateurs.
Angular	Framework web utilisé pour la version web de GuestHub.
Render	Plateforme cloud utilisée pour le déploiement des services et API du projet.

Documents de références

Les documents et sources ci-dessous ont servi de base à la rédaction de ce document des exigences logicielles et à la conception du système GuestHub :

Référence	Description
IEEE Std 830-1998	Norme IEEE pour la spécification des exigences logicielles (Software Requirements Specification).
Documentation interne de l'organisation	Procédures actuelles de gestion des visiteurs et registres papier utilisés à l'accueil.
Guide Apple Developer – SwiftUI (Apple Inc.)	Documentation officielle utilisée pour la conception et le développement de l'interface du système.
Tutoriels et ressources techniques (Xcode, Firebase, etc.)	Documentation utilisée pour l'intégration des services backend et des notifications.
Politique de sécurité de l'organisation	Document interne précisant les règles de confidentialité et de gestion des données des visiteurs.

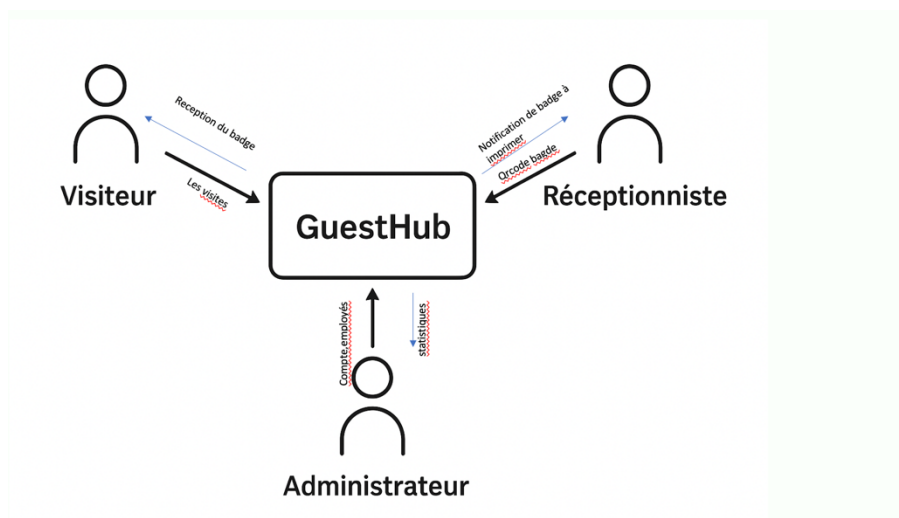
2. DESCRIPTION GÉNÉRALE DU LOGICIEL

Vue d'ensemble des fonctions du produit

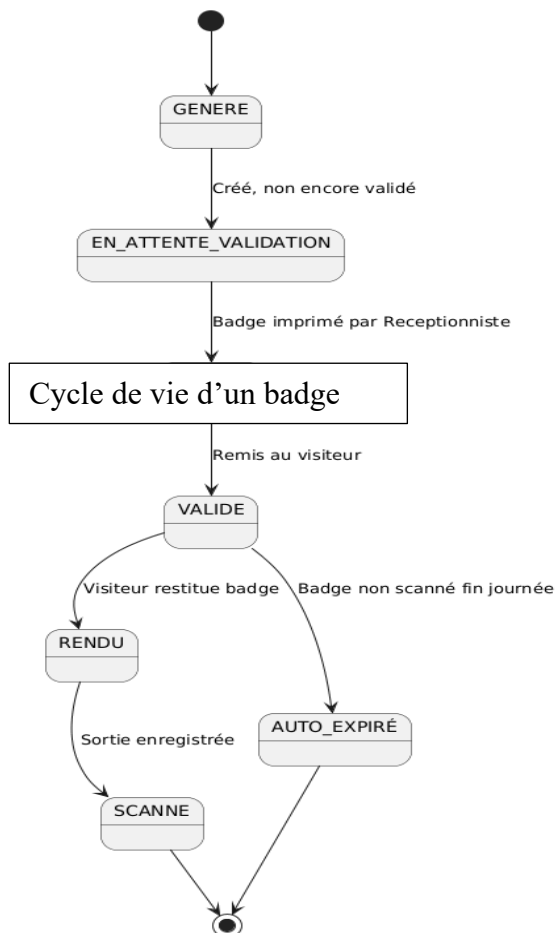
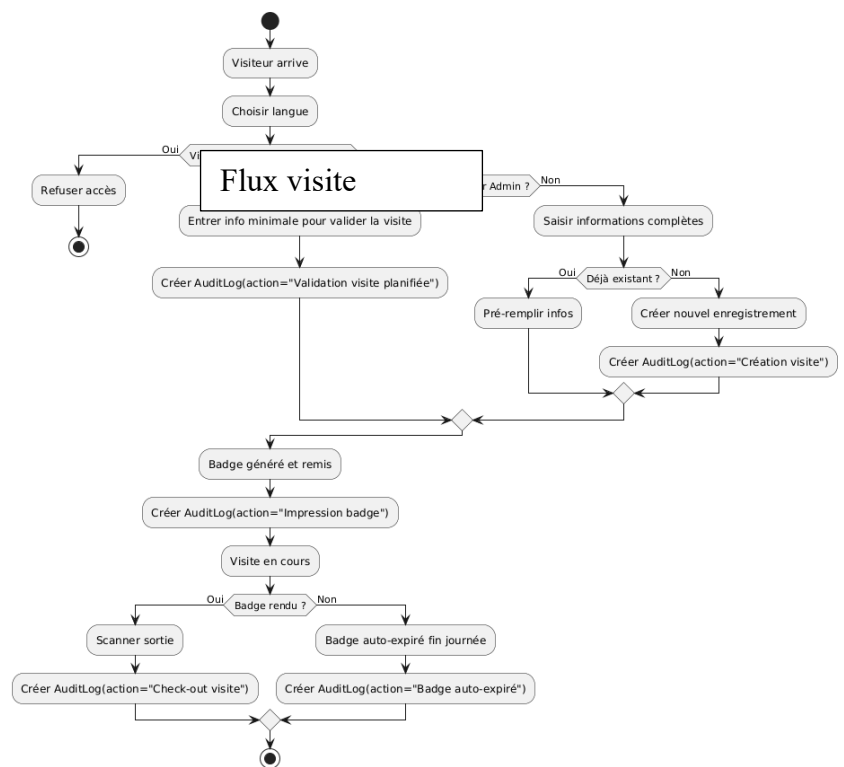
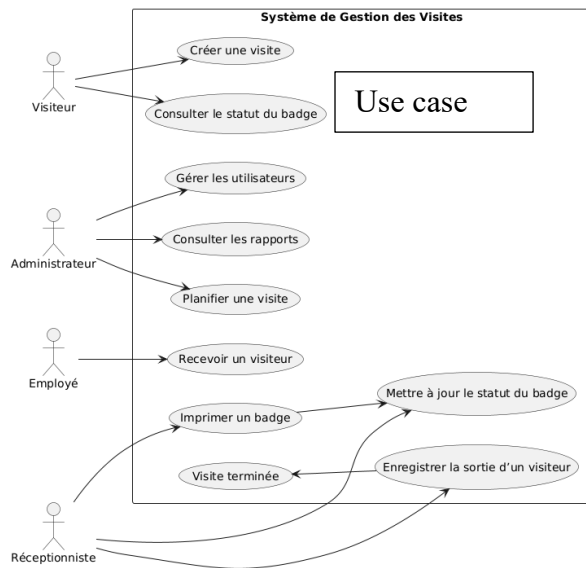
Le système GuestHub offre un ensemble de fonctions majeures pour gérer efficacement les visiteurs d'une organisation :

1. Enregistrement des visiteurs
 - Nouveau visiteur : saisie des informations personnelles, choix de l'employé hôte et création d'une nouvelle visite.
 - Visiteur récurrent : identification via numéro ou email, consultation des informations existantes et validation de visites planifiées.
 - Gestion des visiteurs blacklistés avec blocage automatique.
2. Gestion des badges
 - Génération de badges QR uniques pour chaque visite.
 - Impression des badges par le réceptionniste.
3. Check-in / Check-out des visites
 - Validation de l'arrivée des visiteurs à la réception.
 - Mise à jour de l'état des visites lors du départ.
4. Gestion des utilisateurs et rôles
 - Administrateur : création d'employés, réceptionnistes, planification des visites et consultation des statistiques.
 - Réceptionniste : accès aux fonctions de check-in/check-out et impression des badges.
5. Consultation et exportation des rapports
 - Historique des visites et statistiques de fréquentation.
 - Export possible au format CSV ou PDF pour les administrateurs.

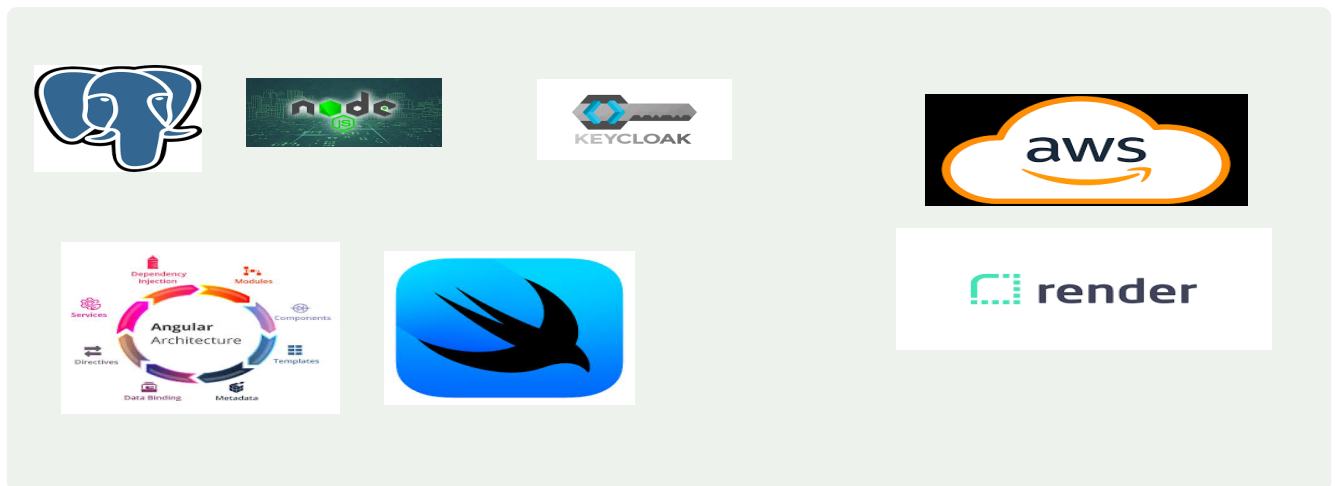
Interfaces externes



Spécifications fonctionnelles



Technologies utilisées



Exigences logiques de bases de données

```
model User {
  id          String    @id @default(cuid())
  login       String    @unique
  email       String    @unique
  password    String
  nom         String
  prenom      String
  role        UserRole  @default(RECEPTIONNISTE)
  isActive    Boolean   @default(true)
  lastLogin   DateTime?
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
  passwordResetRequired Boolean @default(false)
  auditLogs   AuditLog[]
  // Relations
  printedBadges Badge[] @relation("BadgePrintedBy")

  @@map("users")
}

model Department {
  id          String    @id @default(cuid())
  nom         String
  description String?
  isActive    Boolean   @default(true)
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
  employees   Employee[]

  @@unique([nom, isActive])
  @@map("departments")
}
```

```

}

model Employee {
  id      String   @id @default(cuid())
  nom      String
  prenom   String
  email    String   @unique
  telephone String?
  poste    String?
  isActive Boolean   @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  departmentId String?
  department Department? @relation(fields: [departmentId], references: [id])
  visits   Visite[]

  @@map("employees")
}

model Visitor {
  id      String   @id @default(cuid())
  nom      String
  prenom   String
  email    String? @unique
  telephone String?
  entreprise String?
  estBlackliste Boolean @default(false)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  visits   Visite[]

  @@map("visitors")
}

model Visite {
  id      String   @id @default(cuid())
  dateDebut DateTime
  dateFin   DateTime?
  // New fields to explicitly capture time components if needed
  heureDebut DateTime?
  heureFin   DateTime?
  motif      String
  statut     VisitStatus @default(CREEE)
  confirmByVisitor String? // Nom du visiteur qui a confirmé la visite
  confirmedAt DateTime? // Date de confirmation par le visiteur
  createdAt  DateTime   @default(now())
  updatedAt  DateTime   @updatedAt
  visiteurId String
  employeId  String
  // One-to-one relation with Badge
  badge      Badge?
  employe    Employee @relation(fields: [employeId], references: [id])
  visiteur   Visitor  @relation(fields: [visiteurId], references: [id])
}

```

```

    @@map("visits")
}

model Badge {
    id      String  @id @default(cuid())
    qrCode  String  @unique
    status  BadgeStatus @default(GENERATED)
    createdAt  DateTime @default(now())
    updatedAt  DateTime @updatedAt
    dateImpression DateTime?
    printById  String?
    printBy    User?  @relation("BadgePrintedBy", fields: [printById], references: [id])
    // One-to-one: each badge belongs to exactly one visit
    visitId    String  @unique
    visite     Visite  @relation(fields: [visitId], references: [id])

    @@map("badges")
}

model AuditLog {
    id      String  @id @default(cuid())
    action  String
    entityType String
    entityId String?
    details  String?
    dateHeure DateTime @default(now())
    userId  String?
    user    User?  @relation(fields: [userId], references: [id])

    @@map("audit_logs")
}

enum UserRole {
    ADMIN
    RECEPTIONNISTE
}

enum VisitStatus {
    CREEE    // created just now
    PLANIFIEE // future dateDebut
    EN_COURS // in progress (badge printed)
    TERMINEE // closed by checkout
    EXPIREE  // auto-expired (not scanned/closed by end of day)
}

enum BadgeStatus {
    GENERATED // for created/planified visits
    PRINTED    // when printed; set dateImpression & printBy
    CLOSED     // when visit ends or expires
}

model BadgeScanRecord {

```

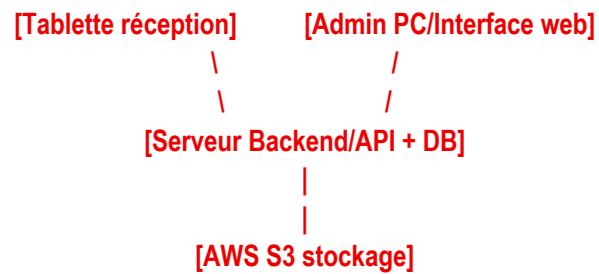
```

id          String  @id @default(cuid())
qrCode      String
action      String  // 'scan' or 'check-out'
visitorName String?
employeeName String?
departmentName String?
visitId     String?
badgeld     String?
scannedBy   String? // User ID who performed the scan
createdAt   DateTime @default(now())

@@map("badge_scan_records")
}

```

Architecture du logiciel



3. DESCRIPTION DÉTAILLÉE

Cette section présente les exigences fonctionnelles et non fonctionnelles du système GuestHub, décrivant les entrées, traitements et sorties pour chacun des acteurs : visiteur, réceptionniste et administrateur.

A. Exigences fonctionnelles

- Accueil du visiteur

Stimulus (Entrée) :

- Le visiteur choisit la langue de son choix (français ou anglais).
- Il sélectionne s'il est nouveau visiteur ou visiteur récurrent.

Traitement :

- Si visiteur récurrent, il saisit son numéro de téléphone ou adresse courriel.
- Le système recherche le visiteur dans la base de données :
 - Si le visiteur est trouvé, il voit ses informations existantes.
 - Si le visiteur est blacklisté, un message d'erreur apparaît indiquant de contacter un responsable.
 - Si le visiteur est non trouvé, il est redirigé vers le formulaire de nouvelle inscription.

Réponse (Sortie) :

- Si le visiteur est trouvé et que son administrateur a planifié une visite, la visite apparaît et il n'a qu'à valider.
- Sinon, il doit créer une nouvelle visite en précisant :
 - le motif de la visite,
 - la durée estimée,
 - le nom de l'employé hôte.
- Une fois la visite validée, le système affiche un message de confirmation invitant le visiteur à se présenter à la réception pour récupérer son badge.

-
- Enregistrement à la réception

Stimulus (Entrée) :

- Le réceptionniste recherche le visiteur dans la liste des arrivées du jour

Traitement :

- Le système vérifie l'état de la visite (planifiée, en attente, annulée).
- Si valide, il marque le visiteur comme "en cours".

- Le badge est imprimé avec les informations du visiteur et le code QR unique.

Réponse (Sortie) :

- Badge imprimé.
- Notification automatique à l'employé hôte indiquant que son visiteur est arrivé.

-
- Fin de visite (Check-out)

Stimulus (Entrée) :

- Le réceptionniste sélectionne la visite active ou scanne le badge.

Traitement :

- Le système met à jour l'heure de départ dans la base de données.

Réponse (Sortie) :

- L'état de la visite devient "terminée".
- Le badge est invalidé.

-
- Gestion des utilisateurs (Administrateur)

Stimulus (Entrée) :

- L'administrateur se connecte à son compte et accède au tableau de bord.

Traitement :

- Il peut :
 - Créer, modifier ou supprimer des employés et les associer à des départements.
 - Gérer les réceptionnistes et leurs droits.
 - Gérer les visiteurs (ajout, suppression, mise à jour, blacklisting).
 - Planifier des visites pour des visiteurs connus.
 - Consulter des statistiques et rapports (visites par jour, par employé, par entreprise, etc.).

Réponse (Sortie) :

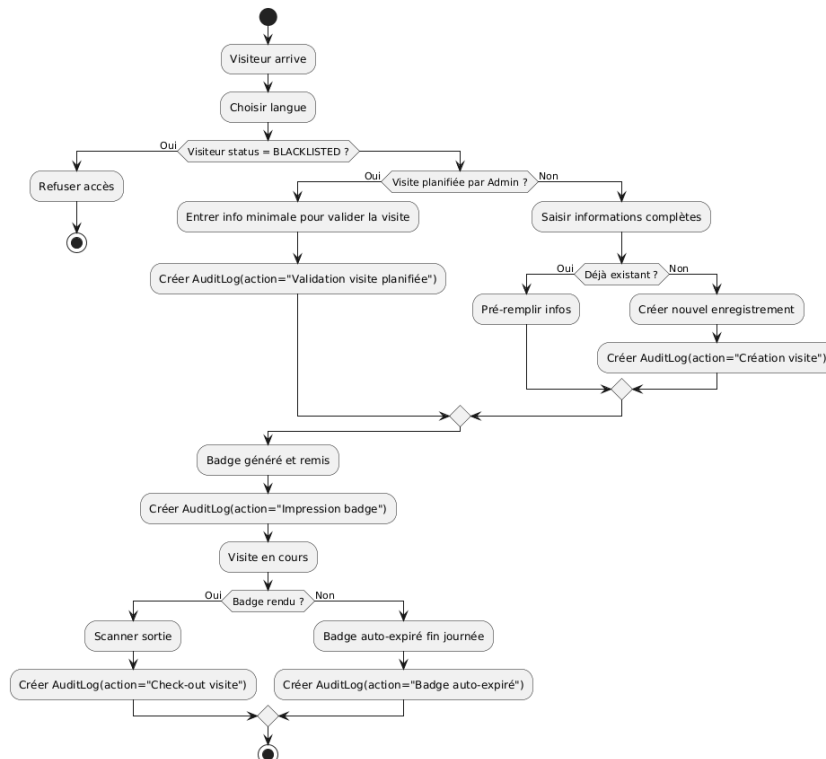
- Confirmation des actions.
- Visualisation des données actualisées dans le tableau de bord.

- Sécurité et rôles

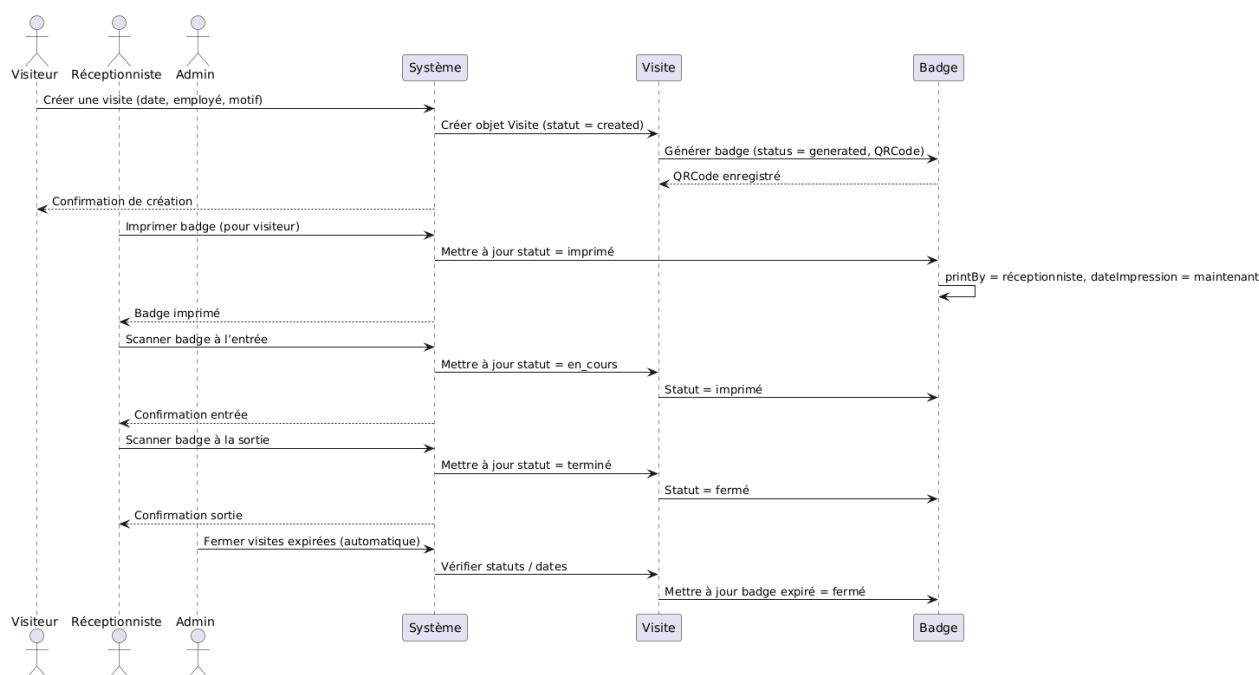
- Visiteur : pas de connexion, accès public restreint à l'enregistrement de visite.
- Réceptionniste : connexion obligatoire, accès à la gestion des arrivées/départs et impression des badges.
- Administrateur : accès complet à toutes les fonctionnalités, y compris la création d'utilisateurs et la consultation des statistiques.

B. Exigences non fonctionnelles

Catégorie	Exigence
Performance	Enregistrement d'une visite en moins de 3 secondes.
Disponibilité	Application disponible 24h/24, 7j/7 (hors maintenance planifiée).
Sécurité	Toutes les communications doivent être chiffrées (HTTPS). Les comptes utilisateurs sont gérés par Keycloak.
Compatibilité	Doit fonctionner sur tablettes, mobiles (iOS/Android) et navigateurs récents.
Ergonomie	Interface simple et intuitive adaptée à un écran tactile d'accueil.
Scalabilité	Capacité à gérer jusqu'à 10 000 enregistrements simultanés sans perte de performance.
Sauvegarde	Sauvegarde quotidienne automatique sur AWS S3.
Langue	Interface bilingue (français et anglais).
Accessibilité	Contrastes suffisants, textes lisibles, navigation fluide sur tous les écrans.



Spécifications fonctionnelles



Exigences logiques de bases de données

Le système **GuestHub** repose sur une base de données relationnelle (**PostgreSQL**) pour stocker et gérer les informations liées aux visiteurs, employés et visites.

- **Types d'informations utilisées :**

- **Visiteur** : nom, prénom, adresse e-mail, téléphone, statut (récurrent, blacklisté, nouveau), historique de visites.
- **Visite** : motif, durée, employé hôte, date et heure d'arrivée/départ, état (planifiée, en cours, terminée).
- **Employé / Réceptionniste / Administrateur** : nom, rôle, département, identifiants de connexion.
- **Badges** : numéro, code QR, état, codevisite.

- **Fréquence d'utilisation :**

- Les informations des visiteurs sont consultées à chaque enregistrement ou validation de visite.
- Les visites planifiées sont vérifiées à chaque arrivée d'un visiteur.
- Les statistiques et rapports sont consultés principalement par les administrateurs.

- **Capacité d'accès :**

- Multi-utilisateurs simultanés (réceptionnistes et administrateurs).
- Accès sécurisé via API REST, authentification Keycloak jwt.

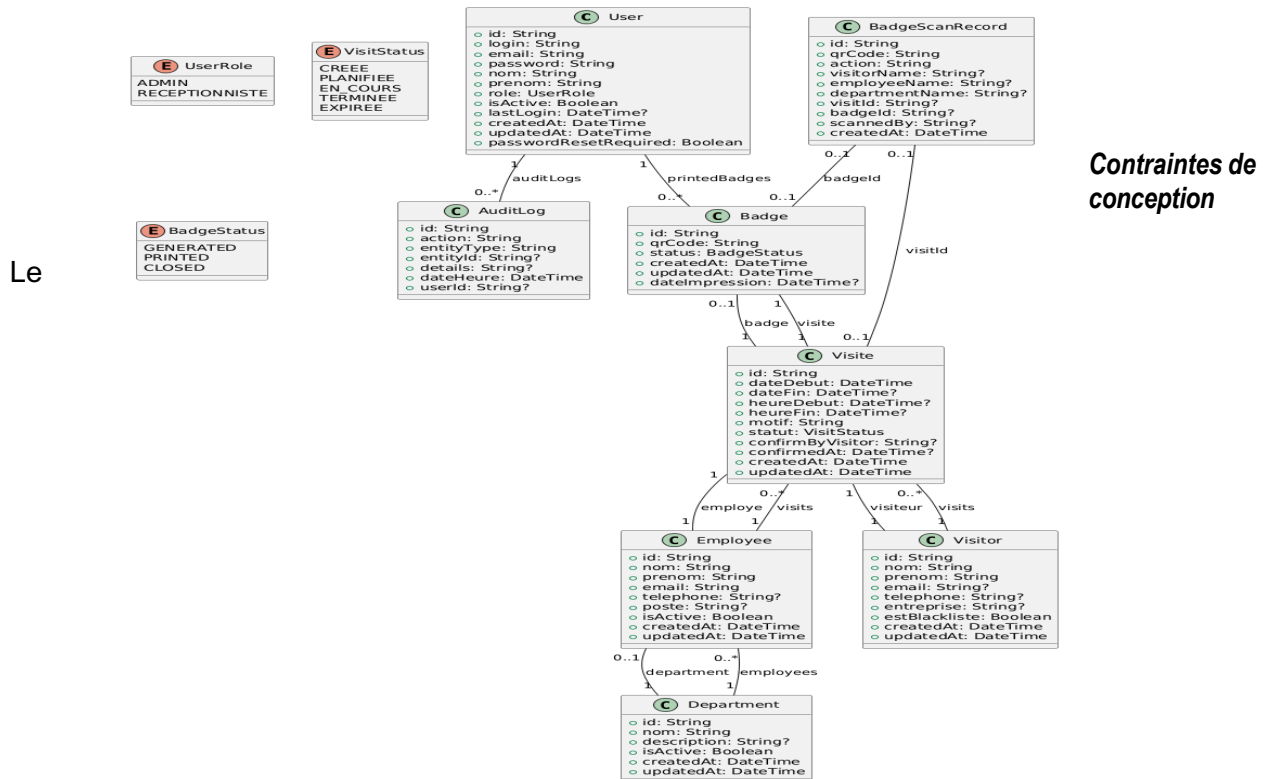
- **Entités et relations :**

- **Visiteur** → **Visite** : un visiteur peut avoir plusieurs visites.
- **Employé** → **Visite** : un employé peut être l'hôte de plusieurs visites.
- **Badge** → **Visite** : un badge est associé à une visite unique.

- **Contraintes d'intégrité :**

- Chaque visite doit être liée à un visiteur et à un employé.

- Les codes QR doivent être uniques.
 - Les visiteurs blacklistés ne peuvent pas créer de nouvelles visites.
- **Exigences de rétention / stockage :**
 - Les données doivent être conservées **au minimum 3 ans** pour des raisons de conformité.
 - Sauvegarde quotidienne automatique sur **AWS S3** avec versioning pour reprise après sinistre.



développement de GuestHub doit respecter les contraintes suivantes :

- **Normes et standards :**
 - Conformité aux standards **IEEE pour le SRS** et aux bonnes pratiques SwiftUI / Node.js / Angular.
 - Respect des normes de sécurité et confidentialité des données personnelles (RGPD).
- **Contraintes matérielles :**
 - Compatible avec tablettes et ordinateurs de réception.
 - Fonctionne sur les systèmes iOS et Android pour l'application mobile.
- **Ergonomie :**
 - Interfaces intuitives pour les visiteurs et la réception.
 - Navigation fluide et guidée pour minimiser les erreurs de saisie.
 - Boutons et champs de saisie adaptés aux écrans tactiles.
- **Autres contraintes :**
 - Impression des badges via imprimantes standard connectées au réseau.
 - Notifications en temps réel aux employés hôtes via l'application.

Exigences non-fonctionnelles

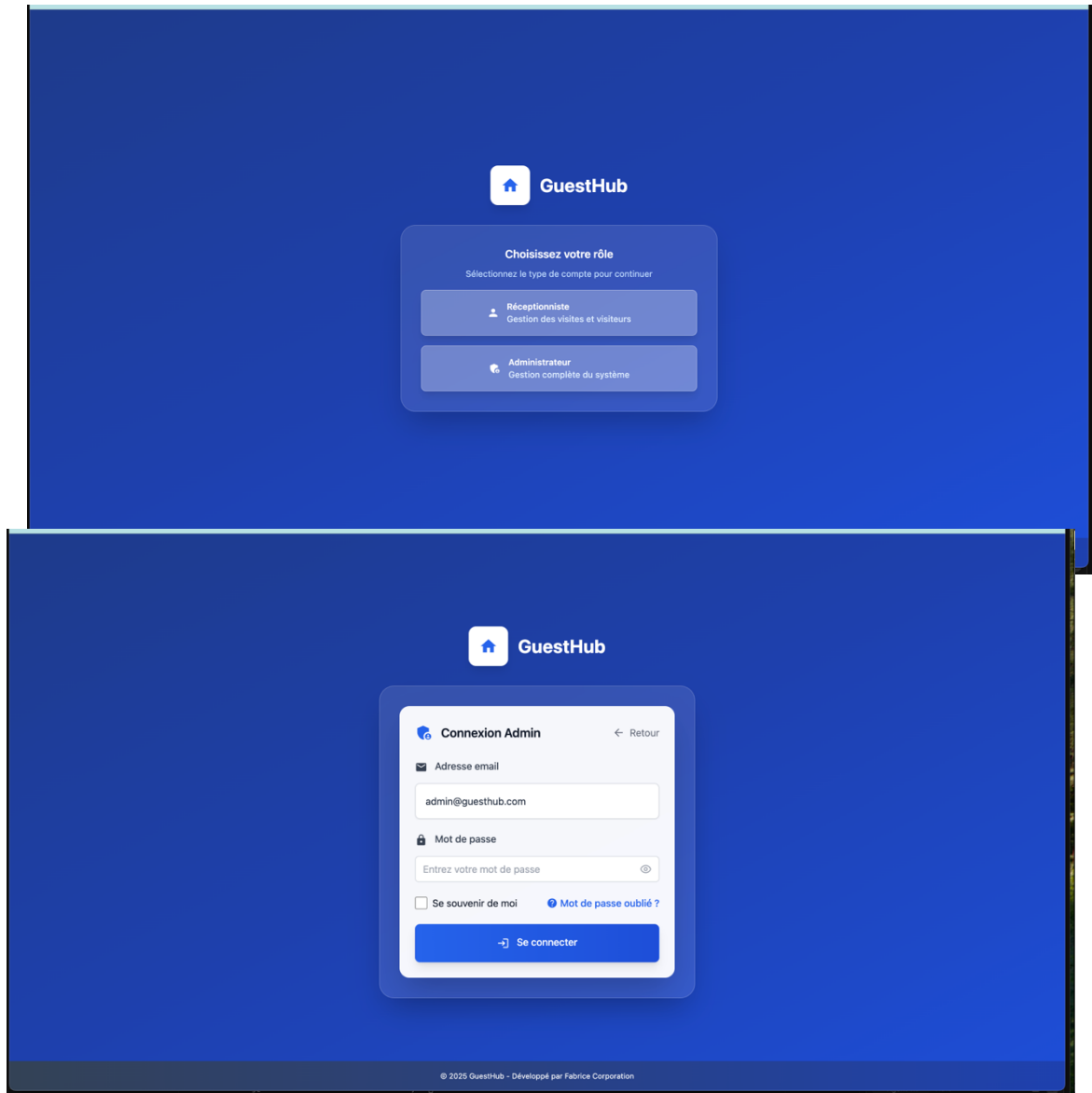
Les exigences non-fonctionnelles définissent les qualités et caractéristiques attendues du système GuestHub :

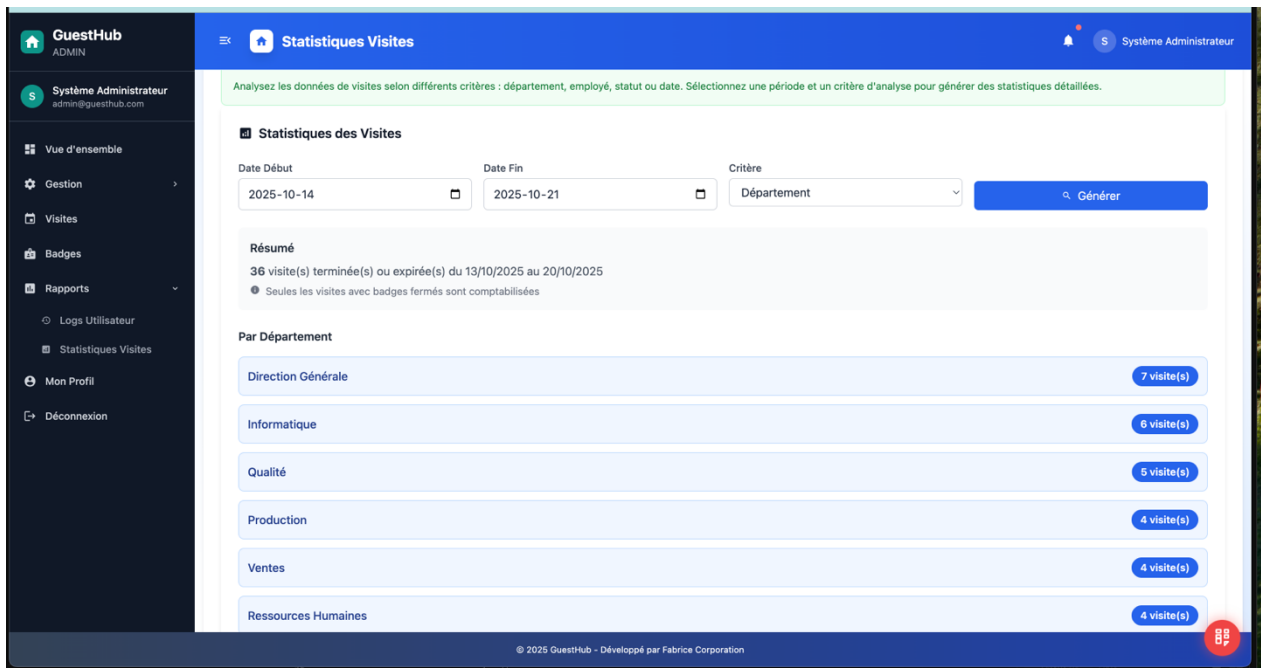
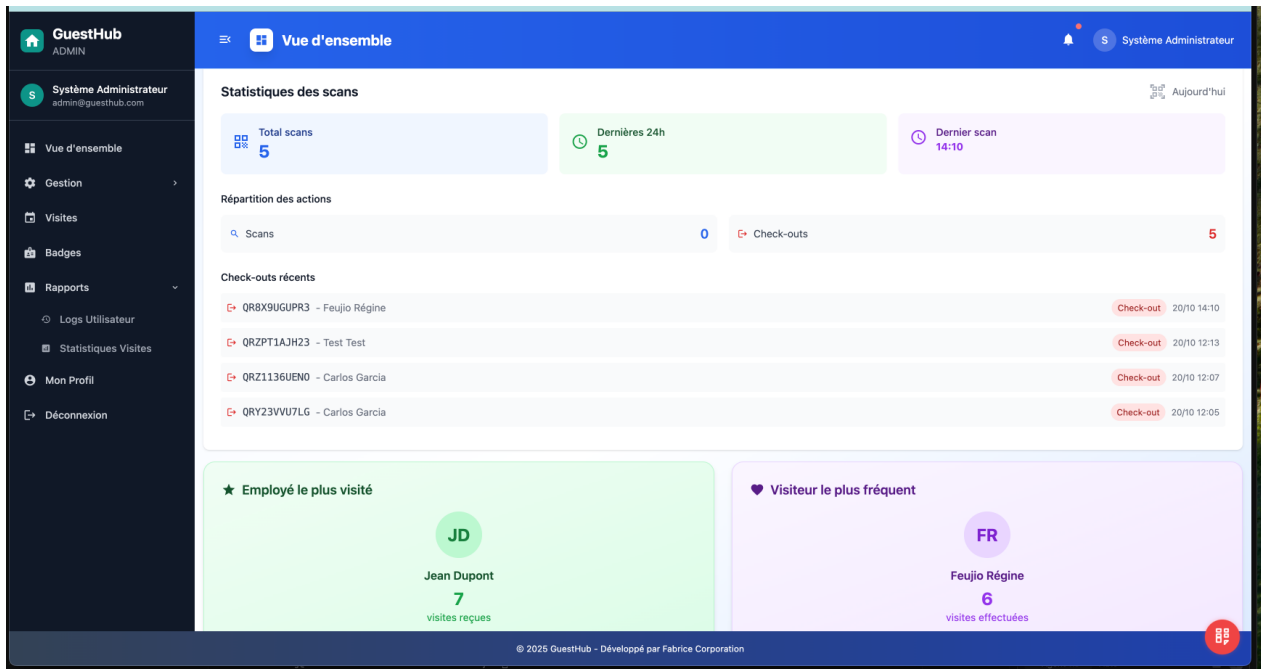
- **Fiabilité et relève :**
 - Le système doit être capable de continuer à fonctionner en cas de panne partielle.
 - Reprise rapide après un crash (moins de 5 minutes).
- **Disponibilité :**
 - L'application doit être disponible 24h/24, 7j/7, hors maintenance planifiée.
- **Sécurité :**
 - Chiffrement des communications (HTTPS).
 - Protection contre les intrusions, modifications ou destructions non autorisées des données.
 - Gestion centralisée des rôles et permissions via Keycloak.
- **Entretien et maintenance :**
 - Mise à jour possible sans interrompre le service global.
 - Journalisation des actions pour faciliter le suivi et le débogage.
- **Modularité et complexité :**
 - Code organisé en modules distincts : interface, backend, API, base de données.
 - Possibilité de faire évoluer le système (ajout de nouvelles fonctionnalités) sans refonte complète.
- **Portabilité :**
 - L'application doit fonctionner sur différents appareils (tablettes, smartphones, PC) et navigateurs modernes.
 - Le backend doit pouvoir être déployé sur différents services cloud (Render, AWS).

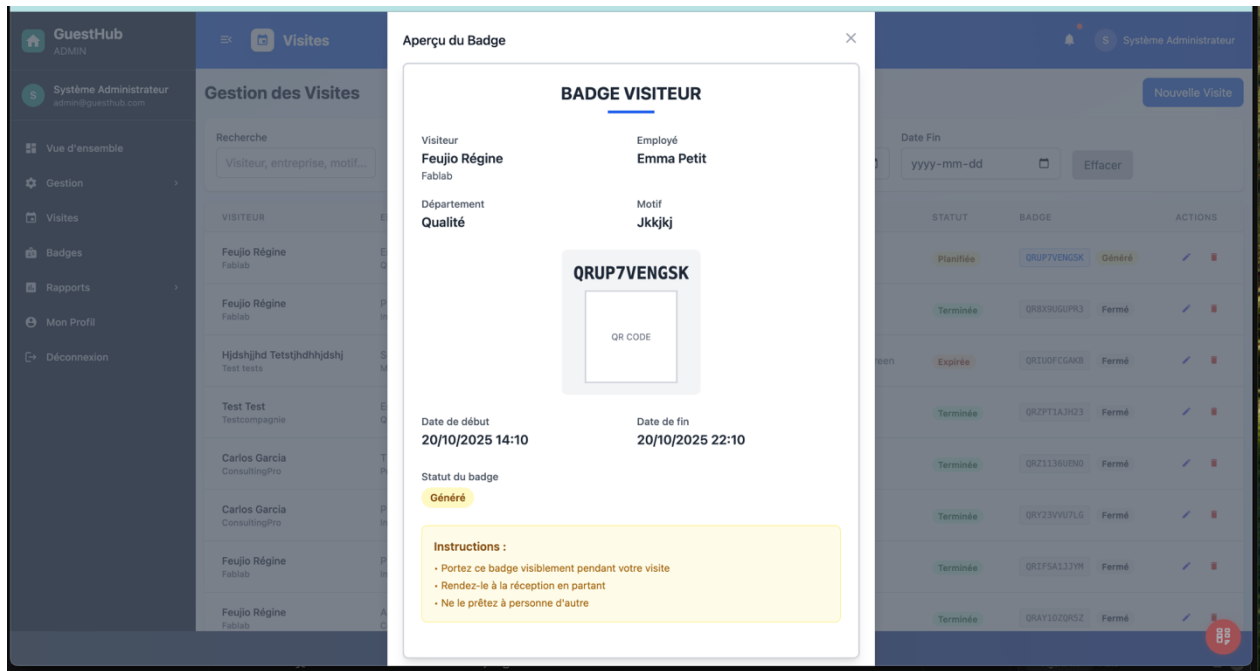
4. INFORMATIONS COMPLÉMENTAIRES

4.1 Annexes

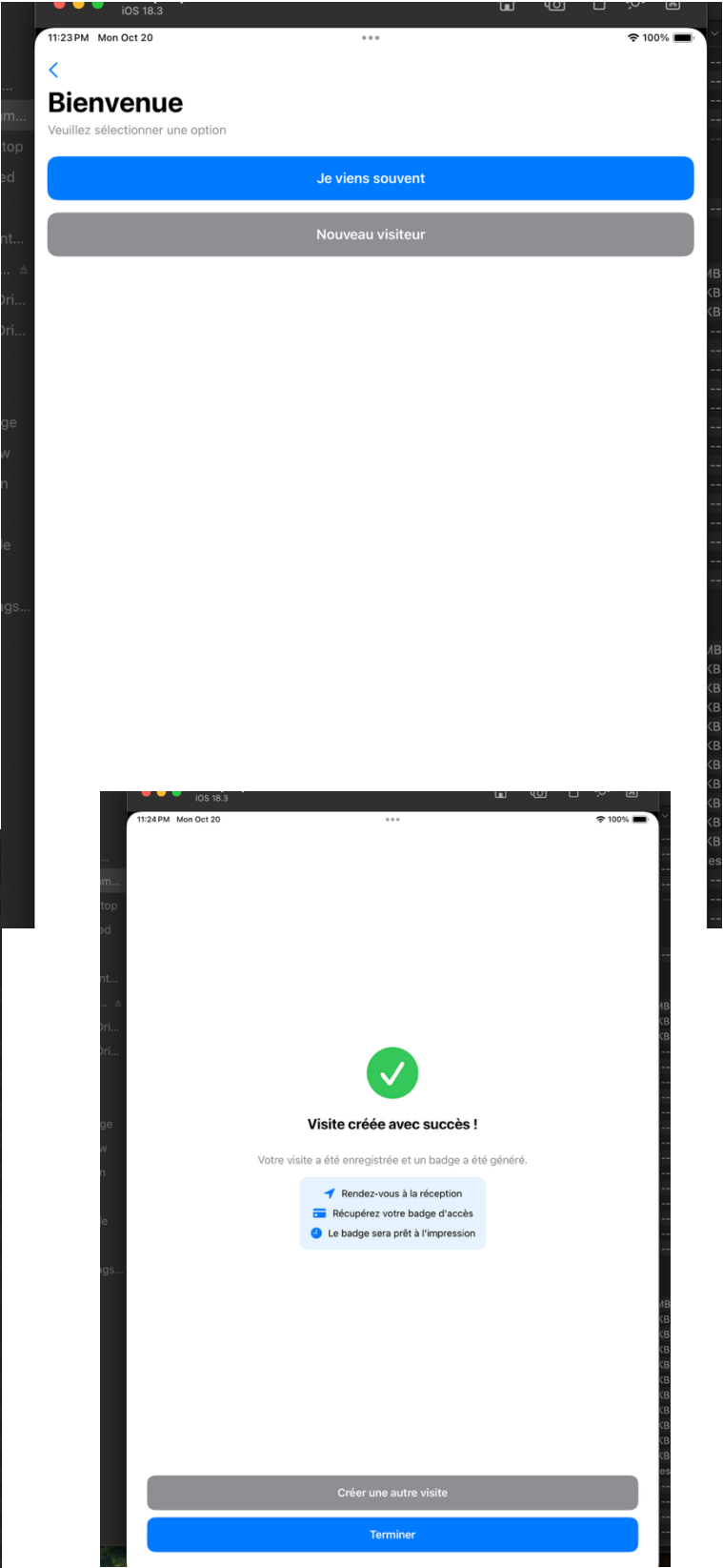
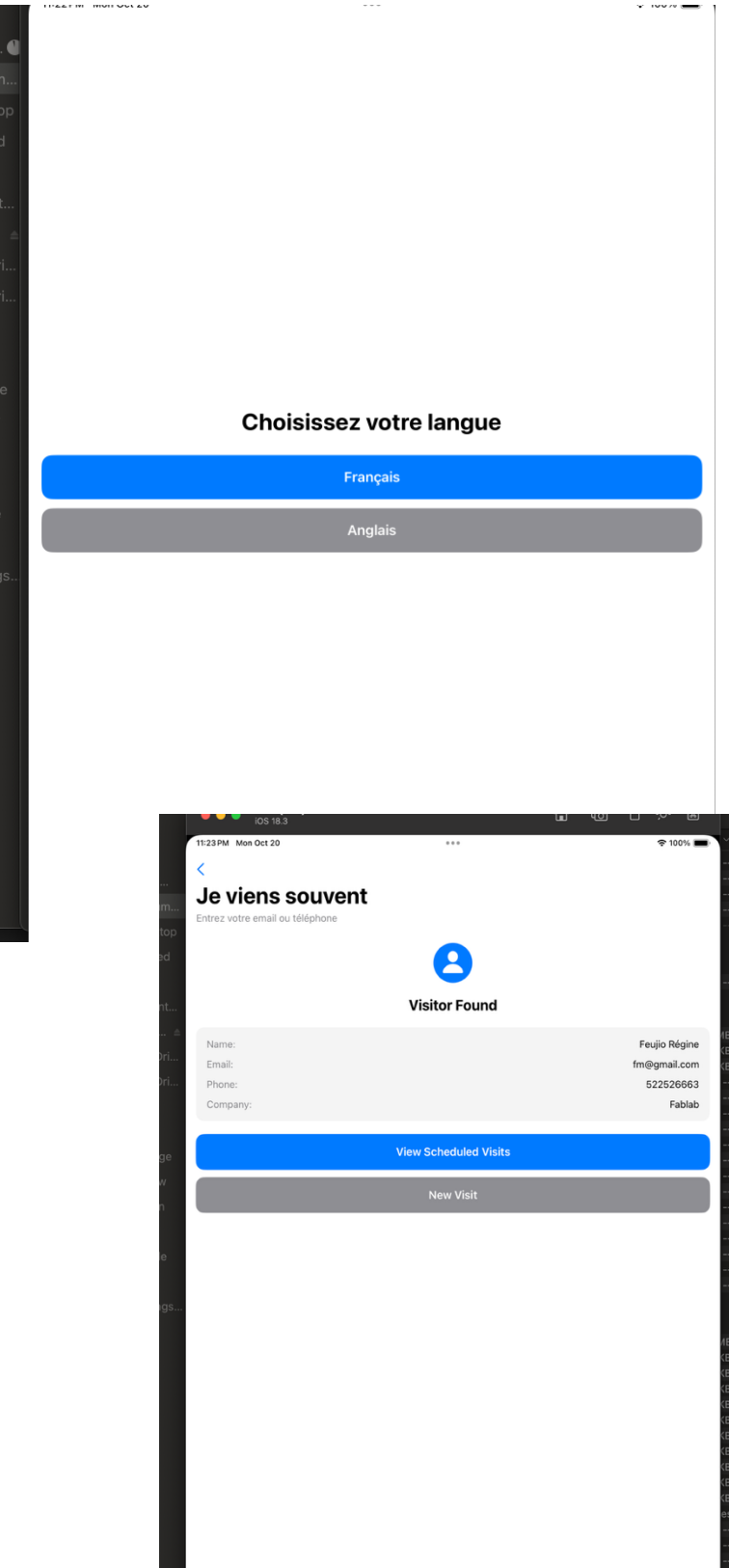
4.2 Quelques interfaces utilisateurs





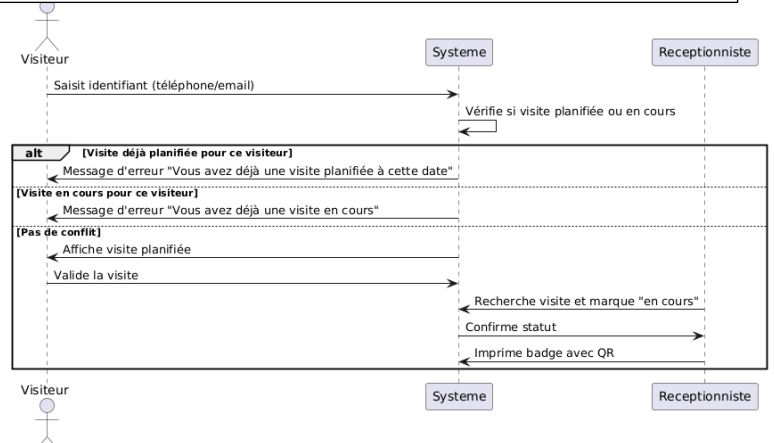
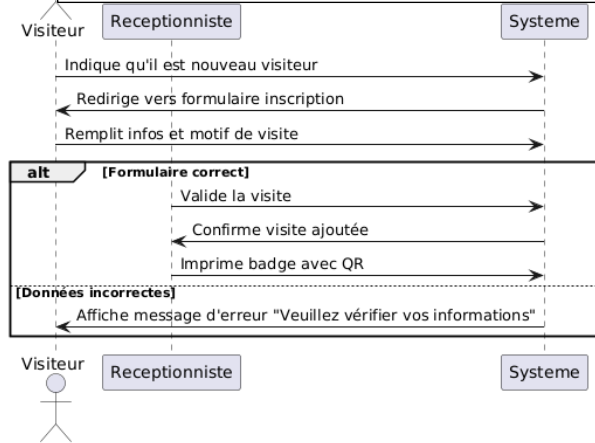


Mobile Sur SwiftUI-----Invité Ipad



GABARIT DE PRÉSENTATION DES CAS D'UTILISATION

Visite planifiée et nouvelle visite ou nouveau visiteur



Checkout du badge

